Type 3 - Simulation (Performance) Study

Shahab Afshar (CprE 558)

CPR E 458/558 Real-Time Systems, Fall 2024

Department of Electrical and Computer Engineering

Iowa State University

Abstract

This project implements and evaluates server-based scheduling algorithms for mixed periodic-aperiodic real-time workloads. A comprehensive discrete-event simulator was developed in Python/Streamlit, implementing four server-based algorithms (Polling Server, Deferrable Server, Sporadic Server, and Background Scheduler) along with foundational scheduling algorithms (RMS, EDF, DMS, LLF). The simulator provides interactive Gantt chart visualizations, schedulability analysis, and performance metrics including CPU utilization, context switches, deadline misses, and aperiodic response times. Evaluation using deterministic benchmark scenarios demonstrates correct implementation of server capacity management policies: Polling Server loses unused capacity, Deferrable Server preserves capacity within periods, and Sporadic Server provides dynamic replenishment for optimal aperiodic response times. The simulator serves as both an educational tool for understanding real-time scheduling concepts and a platform for comparative algorithm analysis.

---

1. Introduction

Real-time systems must satisfy timing constraints in addition to functional correctness. In many practical applications, systems must handle both periodic tasks (with regular, predictable arrivals) and aperiodic tasks (with irregular, event-driven arrivals). Examples include:

- **Automotive systems**: Periodic engine control loops combined with aperiodic driver input handling
- **Industrial automation**: Regular sensor polling alongside sporadic alarm processing
- **Network servers**: Scheduled maintenance tasks with unpredictable client requests

The challenge lies in providing good responsiveness for aperiodic tasks while guaranteeing deadlines for periodic tasks. Server-based scheduling algorithms address this by creating a "virtual" periodic task (the server) that services aperiodic requests using a reserved capacity budget.

This project implements and evaluates four server-based scheduling algorithms from the CprE 458/558 course materials:

1. **Polling Server** - Checks for aperiodic tasks periodically; unused capacity is lost
2. **Deferrable Server** - Preserves unused capacity within the same period
3. **Sporadic Server** - Dynamic replenishment provides best aperiodic response time
4. **Background Scheduler** - Baseline that serves aperiodic tasks only during CPU idle time

The motivation for studying these algorithms stems from their fundamental importance in real-time systems design and their different trade-offs between implementation complexity, schedulability analysis, and aperiodic responsiveness.

---

2. Project Objectives & Scope

2.1. System Model

The system model considers a uniprocessor real-time system with:

- **Periodic tasks**: Tasks with parameters (C , P , D) representing computation time, period, and deadline
- **Aperiodic tasks**: Tasks with parameters (r , C , d) representing arrival time, computation time, and deadline
- **Server**: A virtual periodic task with capacity C and period P that services aperiodic requests

This model appears in embedded systems, RTOS environments, and any application requiring deterministic timing guarantees while handling event-driven workloads.

2.2. Problem Statement

The core problem is: **How do different server capacity management policies affect aperiodic task response times while maintaining periodic task schedulability?**

Specifically: - When no aperiodic tasks are pending, what happens to the server's capacity? - How does capacity replenishment timing affect responsiveness? - What are the trade-offs between implementation complexity and performance?

2.3. Objectives and Scope

Primary Objectives: 1. Implement four server-based scheduling algorithms with correct capacity management 2. Develop an interactive visualization tool for schedule analysis 3. Compare algorithm behavior through deterministic test scenarios 4. Provide educational demonstrations of server scheduling concepts

Scope: - Uniprocessor scheduling (no multiprocessor considerations) - Discrete-time simulation with unit time steps - Focus on server algorithms under RMS priority assignment - Interactive web-based UI for exploration and analysis

---

3. Solution Methodology / Approach

3.1. Algorithms / Protocols / Architectures

3.1.1. Rate Monotonic Scheduling (RMS)

The foundation for server-based scheduling. Tasks are assigned static priorities based on period: shorter period = higher priority. The utilization bound for n tasks is:

$$U = \sum_{i=1}^{n} \frac{C_i}{P_i} \leq n(2^{1/n} - 1)$$

3.1.2. Polling Server

Mechanism: The server is a periodic task with capacity C and period P . At each server invocation: - If aperiodic tasks are waiting: serve them using available capacity - If no aperiodic tasks: **capacity is lost**

Characteristics: - Simple implementation - Non-bandwidth-preserving - Worst aperiodic response time among server algorithms - Maintains RMS schedulability analysis

Implementation: (`scheduler/core/algorithms/combined.py`, class `PollingServerScheduler`)

```python
def _execute_server_slot(self, t: int) -> bool:
    if self.aperiodic_queue and self.server_remaining > 0:
        # Service aperiodic task
        apt = self.aperiodic_queue[0]
        work_done = min(1.0, self.server_remaining, self.aperiodic_remaining[apt.id])
        self.server_remaining -= work_done
        self.aperiodic_remaining[apt.id] -= work_done
        return True
    else:
        # NO APERIODIC TASKS → CAPACITY LOST
        self.server_remaining = 0
        return False
```

3.1.3. Deferrable Server

Mechanism: Unlike Polling, the server preserves its capacity when no aperiodic tasks are available. Capacity can be used at any time during the period.

Characteristics: - Bandwidth-preserving - Better aperiodic response time than Polling - Capacity replenished at period boundaries - May interfere with lower-priority periodic tasks

Key difference from Polling:

```python
def _execute_server_slot(self, t: int) -> bool:
    if self.aperiodic_queue and self.server_remaining > 0:
        # Service aperiodic task (same as Polling)
        # ...
        return True
    else:
        # CAPACITY PRESERVED (Deferrable behavior)
        # Server defers - does NOT run, keeps capacity for later
        return False  # Let periodic tasks run
```

3.1.4. Sporadic Server

Mechanism: Capacity consumed at time t is replenished at time t + P . This dynamic replenishment provides the best aperiodic response times.

Characteristics: - Bandwidth-preserving - Best response time among server algorithms - Maintains RMS utilization bound - Most complex implementation (requires replenishment queue)

Replenishment Logic:

```python
def _consume_capacity(self, amount: float, t: int) -> None:
    self.server_remaining -= amount
    # Schedule replenishment at t + Ps
    replenish_time = t + self.server_period
    self.replenishment_queue.append((replenish_time, amount))
```

3.1.5. Background Scheduler

Mechanism: Aperiodic tasks execute only when the CPU is idle (no periodic tasks ready). This represents the worst-case baseline for aperiodic service.

Characteristics: - Simplest implementation - No interference with periodic tasks - Worst aperiodic response times - Useful as a baseline for comparison

3.2. Illustrative Example

Consider the following workload:

Periodic Tasks: - P1: C=1, P=10, D=10 (10% utilization) - P2: C=1, P=15, D=15 (6.7% utilization)

Aperiodic Tasks: - A1: arrives at t=0, C=8, deadline=50

Server: C =2, P =5

Expected Behavior:

Server Type	A1 Completion Time	Explanation
Polling (C =2)	~t=17	Needs 4 replenishments, capacity lost when A1 not ready
Polling (C =4)	~t=9	Needs 2 replenishments
Polling (C =8)	~t=8	Single replenishment sufficient
Deferrable	Faster than Polling	Capacity preserved for immediate use
Sporadic	Fastest	Dynamic replenishment optimizes response
Background	Slowest	Must wait for CPU idle time

This demonstrates how server capacity directly impacts aperiodic task completion time.

---

4. Implementation/Simulation Architecture

4.1. Software Architecture

The simulator follows a modular architecture with clear separation of concerns:

```
scheduler/
  core/
      task.py              # Task data models (PeriodicTask, AperiodicTask, etc.)
      scheduler_base.py    # Abstract base class with simulation loop
      algorithms/
          rms.py          # Rate Monotonic Scheduling
          edf.py          # Earliest Deadline First
          dms.py          # Deadline Monotonic Scheduling
          llf.py          # Least Laxity First
          combined.py     # Server-based algorithms (Polling, Deferrable, Sporadic, Background)
          precedence.py   # Precedence-constrained variants
          ...
      analysis/
          schedulability.py  # Utilization tests, completion time analysis
      protocols/
           pip_pcp.py      # Resource access protocols
  visualization/
      gantt.py            # Interactive Gantt charts
      metrics_dashboard.py # CPU utilization, context switches
      precedence_graph.py # Task dependency visualization
      timeline_interactive.py # Step-by-step timeline viewer
```

```
configs.py              # Preset task configurations
app.py                  # Streamlit web UI (1000+ lines)
```

4.2. Design Pattern: Template Method

The core design uses the Template Method pattern. `SchedulerBase` implements the complete simulation loop, while concrete schedulers only implement:

```python
class SchedulerBase(ABC):
    @abstractmethod
    def assign_priorities(self) -> None:
        """How to assign priorities to tasks"""

    @abstractmethod
    def get_next_task(self, ready_queue) -> Optional[TaskInstance]:
        """Which task to run next from ready queue"""

    def simulate(self) -> ScheduleResult:
        """Complete simulation loop (implemented in base)"""
```

Server schedulers extend this with custom `simulate()` that handles capacity management:

```python
class ServerScheduler(SchedulerBase):
    @abstractmethod
    def _handle_replenishment(self, t: int) -> None:
        """Server-specific replenishment policy"""

    @abstractmethod
    def _execute_server_slot(self, t: int) -> bool:
        """Server-specific capacity usage policy"""
```

4.3. Technology Stack

Component	Technology	Purpose
Core Logic	Python 3.10+	Simulation engine, data models
Web UI	Streamlit	Interactive dashboard
Visualization	Plotly	Interactive Gantt charts, metrics
Data Handling	Pandas	Task tables, data export
Analysis	NumPy	Numerical computations

4.4. Key Data Structures

TaskInstance - Represents a single instance of a periodic task:

```python
@dataclass
class TaskInstance:
    task_id: str
    instance_number: int
    arrival_time: float
    deadline: float
    remaining_time: float
    completed: bool = False
```

ScheduleEvent - Records simulation events:

```python
@dataclass
class ScheduleEvent:
    time: float
    task_id: str
    event_type: str  # 'start', 'complete', 'preempt', 'deadline_miss', 'replenish'
    details: Optional[Dict] = None
```

ScheduleResult - Complete simulation output:

```python
@dataclass
class ScheduleResult:
    events: List[ScheduleEvent]
    deadline_misses: List[ScheduleEvent]
    cpu_utilization: float
    context_switches: int
```

4.5. UI Features

The Streamlit-based UI provides:

1. **Algorithm Selection**: Category-based organization (Basic, Server-Based, Precedence, Overload, Aperiodic)
2. **Task Configuration**: Editable data grid for task parameters
3. **Server Configuration**: Capacity (C) and Period (P) sliders
4. **Preset Library**: 15+ curated test scenarios from course materials
5. **Schedulability Analysis**: Real-time utilization tests with pass/fail indicators
6. **Interactive Gantt Chart**: Hover for event details, zoom/pan support
7. **Metrics Dashboard**: CPU utilization, context switches, deadline statistics

---

5. Evaluation

5.1. Test Setup

Environment: - Windows 11, Python 3.11 - Streamlit 1.28+, Plotly 5.18+ - Single-machine simulation (discrete-event)

Test Categories: 1. **Unit Tests**: Individual algorithm correctness 2. **Benchmark Scenarios**: Preset configurations from course materials 3. **Server Capacity Demonstrations**: Varying C /P to observe behavior differences

5.2. Performance Metrics

Metric	Definition
CPU Utilization	(Busy time / Duration) × 100%
Context Switches	Count of 'start' events (task begins execution)
Deadline Misses	Count of tasks completing after deadline
Aperiodic Response Time	Completion time - Arrival time

5.3. Test Results

5.3.1. Server Capacity Effect Demo

Workload: P1(C=1,P=10), P2(C=1,P=15), A1(C=8, arrives t=0)

C	P	A1 Response Time	Server Replenishments
2	5	17	4
4	5	9	2
8	5	8	1

Observation: Server capacity directly impacts aperiodic response time. Larger C reduces the number of replenishment cycles needed.

5.3.2. Server Algorithm Comparison

Workload: Basic Server Example (SERVER_EXAMPLE_1) - Periodic: P1(C=2,P=10), P2(C=1,P=8) - Aperiodic: A1(t=3,C=2), A2(t=8,C=1), A3(t=15,C=2) - Server: C =2, P =5

Algorithm	Avg Response Time	Behavior
Polling	Moderate	Capacity lost when no aperiodic tasks
Deferrable	Good	Capacity preserved for immediate use
Sporadic	Best	Dynamic replenishment optimizes timing
Background	Worst	Only runs during CPU idle

5.3.3. Schedulability Analysis Validation

RMS Utilization Test: - Task set: T1(C=2,P=4), T2(C=1,P=8) - U = 2/4 + 1/8 = 0.625 - Bound (n=2): 2(2^0.5 - 1) = 0.828 - Result: **SCHEDULABLE** (0.625 0.828)

EDF Utilization Test: - Task set: T1(C=1,P=3), T2(C=4,P=6) - U = 1/3 + 4/6 = 1.0 - Result: **SCHEDULABLE** (U 1.0)

5.4. Visualization Examples

The simulator produces interactive Gantt charts showing: - Task execution intervals (color-coded by task) - Preemption points - Deadline markers - Server capacity events (replenish, deferred, capacity_lost)

---

6. Conclusions

6.1. Summary

This project successfully implemented a comprehensive real-time scheduling simulator with:

- **4 Server-Based Algorithms**: Polling, Deferrable, Sporadic, and Background schedulers with correct capacity management
- **4 Basic Algorithms**: RMS, EDF, DMS, LLF for foundational scheduling
- **Interactive Visualization**: Plotly-based Gantt charts with detailed event information
- **Schedulability Analysis**: Utilization tests for RMS, EDF, and DMS
- **Educational Presets**: 15+ curated scenarios demonstrating algorithm behavior

6.2. Key Findings

1. **Server Capacity Impact**: Larger server capacity (C) directly reduces aperiodic response times by requiring fewer replenishment cycles
2. **Algorithm Trade-offs**:

- Polling Server: Simplest but loses unused capacity
- Deferrable Server: Better responsiveness, preserves capacity
- Sporadic Server: Best responsiveness, maintains schedulability
- Background: Baseline with worst aperiodic performance

3. **Implementation Complexity**: The Template Method pattern reduced code duplication for basic algorithms (RMS, EDF, DMS, LLF). Server-based algorithms implement custom simulation loops to handle capacity management, demonstrating appropriate design flexibility.

6.3. Recommendations for Future Work

1. **Total Bandwidth Server (TBS)**: EDF-based server with dynamic deadline assignment
2. **Multi-Server Configurations**: Multiple servers with different priorities
3. **Statistical Analysis**: Random task generation using UUniFast algorithm
4. **Resource Protocols**: Integration of Priority Inheritance/Ceiling protocols

---

Self-Assessment of Project Completion

Project Learning Objectives	Status	Pointers
Self-contained description of project goal, scope, and requirements	Fully Completed	Section 2, pages 2-3
Self-contained description of solutions (algorithms/protocols)	Fully Completed	Section 3, pages 3-6
Adequate description of implementation details	Fully Completed	Section 4, pages 6-8
Testing and evaluation - test cases, metrics, results	Fully Completed	Section 5, pages 8-10
Overall Project Success Assessment	**Fully Successful**	

---

7. References

[1] B. Sprunt, L. Sha, and J. P. Lehoczky, "Aperiodic task scheduling for hard-real-time systems," *Real-Time Systems*, vol. 1, no. 1, pp. 27-60, 1989.

[2] J. P. Lehoczky, L. Sha, and J. K. Strosnider, "Enhanced aperiodic responsiveness in hard real-time environments," in *Proc. IEEE Real-Time Systems Symposium*, 1987, pp. 261-270.

[3] M. Spuri and G. C. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, no. 2, pp. 179-210, 1996.

[4] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46-61, 1973.

[5] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York: Springer, 2011.

[6] E. Bini and G. C. Buttazzo, "Measuring the performance of schedulability tests," *Real-Time Systems*, vol. 30, no. 1-2, pp. 129-154, 2005.

Prepared by: Shahab Afshar **Date:** December 2024 **Instructor:** Dr. G. Manimaran **Department:** Electrical and Computer Engineering, Iowa State University